```
# Importing Libraries
# Numpy and Pandas
import numpy as np
import pandas as pd

# Data Visualization
import matplotlib.pyplot as plt
# Pre-Processing Module
from sklearn.preprocessing import MinMaxScaler

# Keras Modules
from keras.models import Sequential
from keras.layers import *
from keras.layers import LSTM, Dropout

# Importing dataset
tesla = pd.read_csv("/content/Tesla.csv")

# Printing dataset
print(tesla)

# Data Splitting
training = tesla.iloc[:800, 1:2].values
testing = tesla.iloc[800:, 1:2].values

# Data Normalization
mm_scale = MinMaxScaler(feature_range=(0, 1))
training_scaled = mm_scale.fit_transform(training)

# Data structure with 60 time steps and 1 output
x_train = []
y_train = []
for i in range(60, 800):
    x_train.append(training_scaled[i-60:i, 0])
    y_train.append(training_scaled[i, 0])
x_train, y_train = np.array(x_train), np.array(y_train)
x_train = np.reshape(x_train, (x_train.shape[0], x_train.shape[1], 1))

# Model Architecture
model = Sequential()

# Building Model
model.add(LSTM(units=50, return_sequences=True, input_shape=(x_train.shape[1], 1)))
model.add(Dropout(0.2))
model.add(LSTM(units=50, return_sequences=True))
model.add(Dropout(0.2))
model.add(LSTM(units=50, return_sequences=True))
model.add(Dropout(0.2))
model.add(LSTM(units=50))
model.add(Dropout(0.2))
model.add(Dense(units=1))

# Compiling the model
model.compile(optimizer='adam', loss='mean_squared_error')

# Fitting values to model
model.fit(x_train, y_train, epochs=100, batch_size=32)

# Preparing Test Data
train_data = tesla.iloc[:800, 1:2]
test_data = tesla.iloc[800:, 1:2]
total_data = pd.concat((train_data, test_data), axis=0)
inputs = total_data[len(total_data) - len(test_data) - 60:].values
inputs = inputs.reshape(-1, 1)
inputs = mm_scale.transform(inputs)

x_test = []
for i in range(60, 519):
    x_test.append(inputs[i-60:i, 0])
x_test = np.array(x_test)
x_test = np.reshape(x_test, (x_test.shape[0], x_test.shape[1], 1))

# Prediction
stock_price_predicted = model.predict(x_test)
stock_price_predicted = mm_scale.inverse_transform(stock_price_predicted)
```

```
# Visualization
plt.figure(figsize=(12, 6))
plt.plot(tesla.loc[800:, "Date"], test_data.values, color="red", label="Actual Price")
plt.plot(tesla.loc[800:, "Date"], stock_price_predicted, color="blue", label="Predicted Price")
plt.xticks(np.arange(0, 459, 50))
plt.title('TESLA Stock Price Prediction')
plt.xlabel('Time')
plt.ylabel('Stock Price')
plt.legend()
plt.show()
```